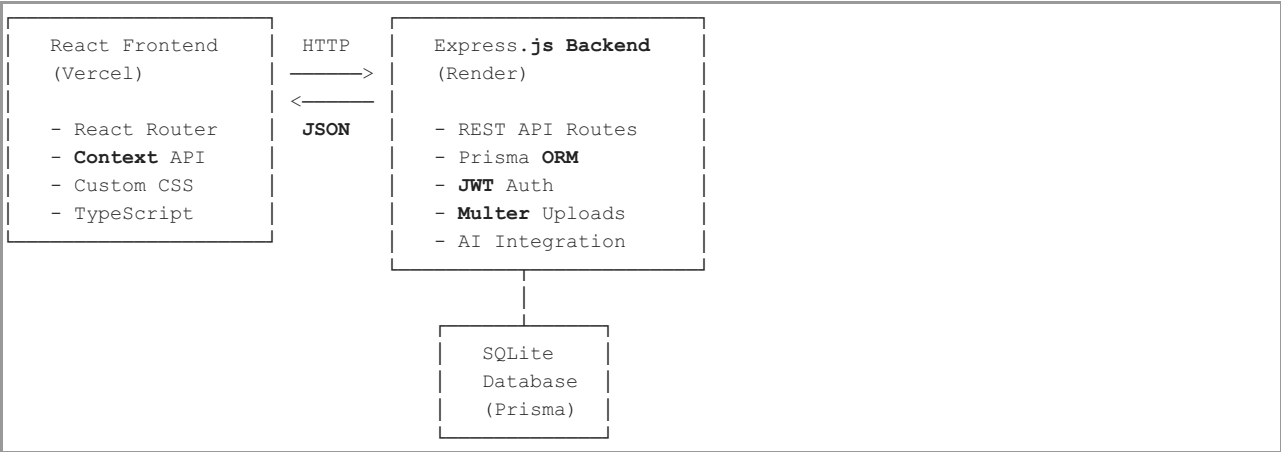


PathFinder — System Architecture Document

1. High-Level Architecture

PathFinder follows a classic **Client-Server architecture** with clear separation between the frontend presentation layer and the backend API layer.



Communication Protocol

- **Protocol:** HTTP/HTTPS
- **Data Format:** JSON (application/json)
- **Authentication:** Bearer token in Authorization header
- **File Upload:** multipart/form-data via Multer middleware

2. Frontend Architecture

2.1 Component Hierarchy

The frontend uses a component-based architecture organized into pages and reusable components.

```
App.tsx (Root)
├── AuthContext (Global State)
├── ThemeContext (Global State)
├── Public Routes
│   ├── Landing.tsx
│   ├── Login.tsx
│   ├── Register.tsx
│   ├── ForgotPassword.tsx
│   └── ResetPassword.tsx
├── Protected Routes (Student)
│   ├── DashboardLayout.tsx (Sidebar Wrapper)
│   │   ├── StudentDashboard.tsx
│   │   ├── StudentProfile.tsx
│   │   ├── StudentApplications.tsx
│   │   ├── SearchPage.tsx
│   │   ├── BookmarkedOpportunities.tsx
│   │   ├── AIAssistant.tsx
│   │   ├── NotificationCenter.tsx
│   │   ├── Tickets.tsx
│   │   └── TicketDetails.tsx
├── Protected Routes (Organization)
│   ├── DashboardLayout.tsx (Sidebar Wrapper)
│   │   ├── OrganizationDashboard.tsx
│   │   ├── OrganizationProfile.tsx
│   │   ├── PostOpportunity.tsx
│   │   ├── OrganizationApplicants.tsx
│   │   ├── NotificationCenter.tsx
│   │   ├── Tickets.tsx
│   │   └── TicketDetails.tsx
└── Protected Routes (Admin)
    └── AdminDashboard.tsx
```

2.2 State Management

- **AuthContext:** Stores JWT token, user object (id, email, role), login/logout functions. Persisted to localStorage.
- **ThemeContext:** Stores dark/light mode preference. Toggled via UI switch. Persisted to localStorage.

2.3 Design System

The CSS design system is defined entirely in `index.css` using CSS custom properties (variables). Key design tokens:

- Color palette (primary, secondary, background, surface, text colors)
- Glassmorphism effects (backdrop-filter: blur)
- Border radius tokens
- Shadow depths
- Typography scale
- Responsive breakpoints
- Animation timing functions

3. Backend Architecture

3.1 Application Structure

```

backend/
├── index.ts                # Express app setup, middleware, route mounting
├── middleware/
│   └── auth.ts            # JWT verification + role authorization middleware
├── lib/
│   ├── prisma.ts         # Singleton Prisma client instance
│   └── notifications.ts   # Helper functions for creating notifications
├── routes/
│   ├── auth.ts           # Registration, login, password reset
│   ├── student.ts        # Student profile CRUD
│   ├── organization.ts   # Organization profile CRUD
│   ├── opportunities.ts  # Opportunity CRUD + filtering
│   ├── applications.ts   # Application submission, status updates, matching
│   ├── ai.ts             # AI generators, resume parsing, recommendations
│   ├── search.ts         # Search and filter endpoint
│   ├── notifications.ts  # Notification retrieval and management
│   ├── admin.ts          # Admin analytics, user management, system health
│   ├── stats.ts          # Dashboard statistics endpoint
│   └── tickets.ts        # Support ticket CRUD and messaging
├── prisma/
│   ├── schema.prisma     # Database schema definition
│   └── dev.db            # SQLite database file
└── uploads/              # Uploaded files (resumes, images)

```

3.2 Middleware Pipeline

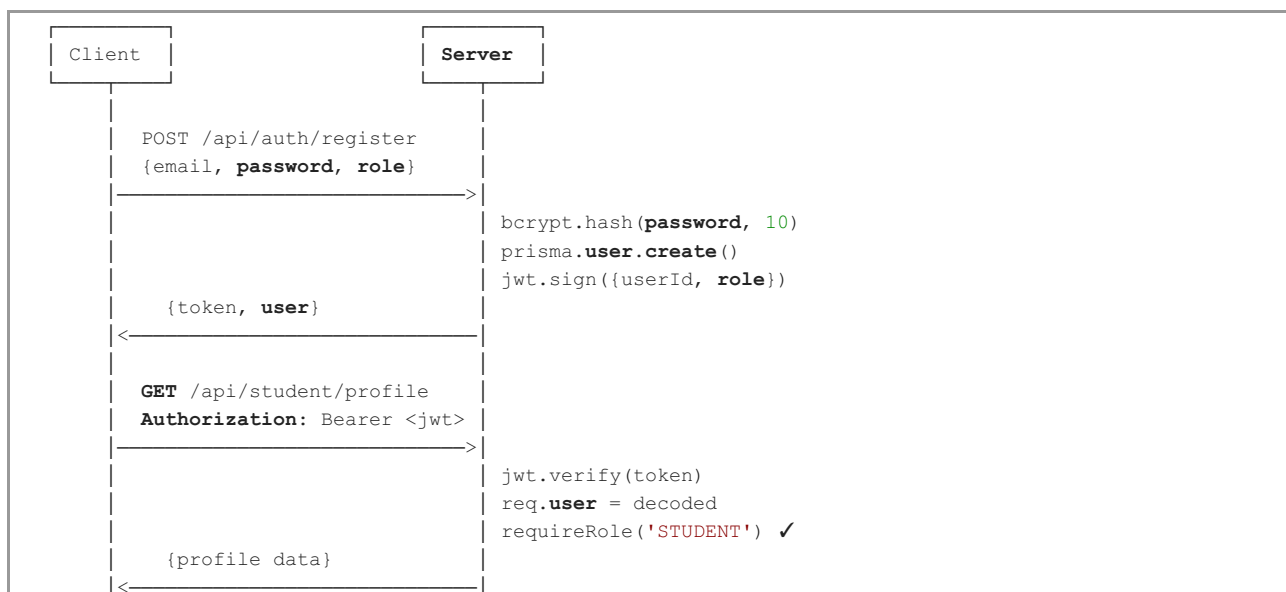
Every incoming HTTP request passes through the following middleware chain:

1. **CORS** — Validates the request origin against allowed domains.
2. **express.json()** — Parses JSON request bodies.
3. **Rate Limiter** — Applied specifically to /api/auth routes (5 req/15 min).
4. **authenticate** — Verifies the JWT token from the Authorization header and attaches the decoded user object to the request.
5. **requireRole(role)** — Checks that the authenticated user has the required role (STUDENT, ORGANIZATION, or ADMIN).
6. **Route Handler** — Executes the business logic for the specific endpoint.

3.3 File Upload Pipeline

1. Client sends a multipart/form-data POST request to /api/upload.
2. Multer middleware intercepts the request, validates file type and size.
3. File is saved to the /uploads directory with a unique filename (timestamp + random).
4. Server responds with the file URL.
5. For retrieval: images are served publicly; documents require JWT authentication.

4. Authentication Flow



5. AI Matching Workflow

The match scoring operates asynchronously after application submission:

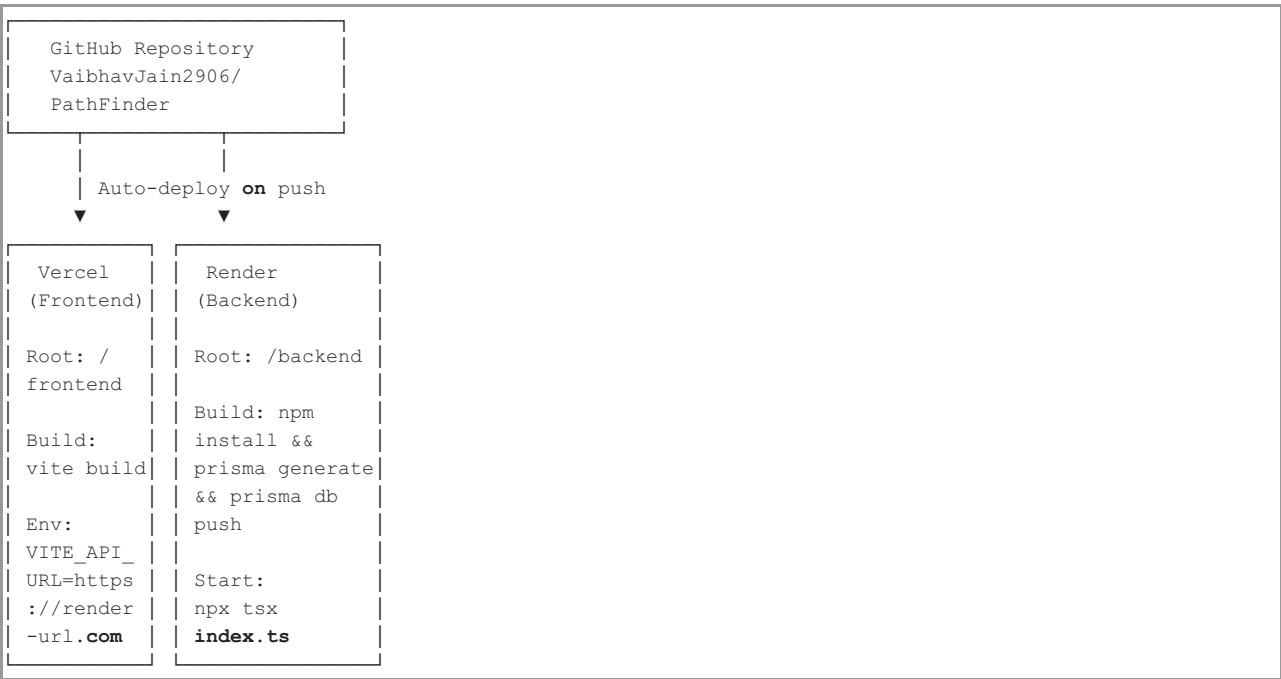
1. Student submits application → immediate 201 response to client.
2. Background async function fires:
 - a. If external AI API key is available: sends structured prompt with student profile + opportunity details → receives JSON with matchScore and matchReason.
 - b. If no API key (mock mode): generates a score between 60-95 with contextual reason text based on skills overlap.
3. Application record is updated with matchScore and matchReason.
4. Organization sees the score when they next view the applicant list.

6. Notification Flow

Notifications are generated server-side and stored in the database:

1. **Trigger Events:**
 - Application status change → Student receives notification
 - New application received → Organization receives notification
2. **Storage:** Notification record created in the Notification table with title, message, type, and read status.
3. **Retrieval:** Frontend polls GET /api/notifications on page load.
4. **Display:** Unread count shown as a badge on the bell icon in the sidebar.

7. Deployment Architecture



8. Security Architecture

Defense in Depth Layers

1. **Network Layer:** CORS restricts origins; rate limiting protects against brute force.
2. **Authentication Layer:** JWT tokens with 24-hour expiry; bcrypt password hashing.
3. **Authorization Layer:** Role-based middleware prevents unauthorized access.
4. **Data Layer:** Prisma ORM prevents SQL injection; input is validated before database operations.
5. **File Layer:** Upload validation restricts file types and sizes; document access requires authentication.
6. **Application Layer:** Error handlers prevent stack trace leakage to clients.

9. Folder Structure (Complete)

```
PathFinder/
├── .gitignore
├── README.md
├── Guide.md
├── Guide.pdf
├── Documents/           # This documentation suite
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   │   ├── DashboardLayout.tsx
│   │   │   └── ProtectedRoute.tsx
│   │   ├── context/
│   │   │   ├── AuthContext.tsx
│   │   │   └── ThemeContext.tsx
│   │   ├── pages/      # 20 page components
│   │   ├── App.tsx     # Route definitions
│   │   ├── main.tsx    # React entry point
│   │   └── index.css    # Design system
│   ├── index.html
│   ├── tsconfig.json
│   ├── vite.config.ts
│   └── package.json
├── backend/
│   ├── middleware/
│   │   └── auth.ts
│   ├── lib/
│   │   ├── prisma.ts
│   │   └── notifications.ts
│   ├── routes/         # 11 route modules
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── dev.db
│   ├── uploads/
│   ├── index.ts
│   ├── tsconfig.json
│   └── package.json
```

10. Scalability Strategy

While the current MVP uses SQLite, the architecture is designed for easy scaling:

1. **Database Migration:** Change one line in `schema.prisma` from `provider = "sqlite"` to `provider = "postgresql"` and update the `DATABASE_URL`. Prisma handles the rest.
2. **Horizontal Scaling:** The stateless JWT authentication allows multiple backend instances behind a load balancer.
3. **File Storage:** Currently local disk can be swapped to cloud storage (AWS S3, Cloudinary) by changing the Multer destination.
4. **Caching:** Redis can be introduced for session caching and rate limiting without modifying application logic.

End of System Architecture Document